

Information Security Education & Awareness (ISEA) Phase III
Tezpur University

INTERNSHIP REPORT

Assignment 8

Application Optimization, Scalability, and Reliability

Student Name	Parinita Devi
Roll / ID No.	CS-BTC25-39
Course	ISEA Phase III Networking Internship
Department	Computer Science & Engineering, Tezpur University
GitHub Repository	https://github.com/parinita-devi

Submitted as part of the ISEA Phase III Internship Program

1. Objective

The goal of this assignment is to optimize the existing GUI-based multi-client chat application (from Assignment 7) by enhancing its scalability, network reliability, maintainability, and resource management. Key objectives include migrating parameter management to dynamic JSON configurations, replacing unbounded thread generation with structured thread pools, adding auto-reconnect and heartbeat mechanisms, and verifying system metrics under multi-client load in Mininet.

2. Scalability Improvements

To support 10 or more concurrent client connections without crashes or thread starvation, the following measures were implemented:

- **Thread Pool Executor:** Replaced raw threading.Thread instantiation per connection with Python's concurrent.futures.ThreadPoolExecutor in server.py.
- **Resource Bounding:** Set a maximum worker thread limit to prevent system resource exhaustion during spikes in connection attempts.
- **Session Tracking:** Implemented thread-safe dictionary structures to track active sessions cleanly, automatically clearing state upon client disconnections.

3. Reliability Improvements

Several mechanisms were integrated into server.py and client_gui.py to handle network anomalies gracefully:

- **Socket Timeouts:** Added non-blocking socket timeouts (socket.settimeout()) to catch hanging connections and release locked sockets.
- **Automatic Reconnection:** Configured the GUI client to catch lost socket signals and trigger auto-reconnection loops without crashing the UI.
- **Heartbeat & Inactivity Detection:** Server detects missing client keep-alive signals, cleans up inactive client sessions, and frees associated memory/buffers.
- **Graceful Shutdown:** Implemented clean signal handling (SIGINT) on the server to terminate worker threads and safely close active sockets without leaving ports bound in TIME_WAIT.

3.1 Auto-Reconnect in Action

When a client's connection to the server is lost, the GUI automatically detects the disconnection and enters a reconnection loop, notifying the user via a status dialog rather than crashing the interface.

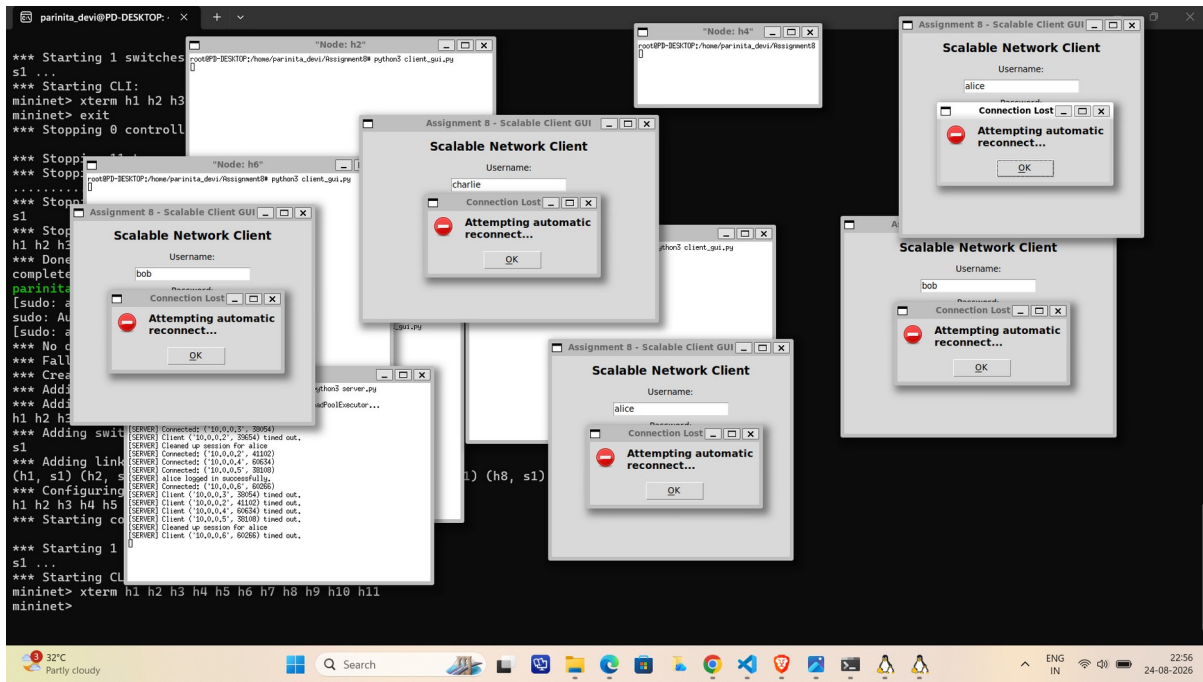


Figure 1: Multiple client GUIs displaying “Connection Lost – Attempting automatic reconnect...” after simulated network interruption

4. Configuration Management

Hardcoded connection variables were decoupled from executable source code and centralized into config.json:

```
{
  "SERVER_IP": "10.0.0.1",
  "PORT": 12345,
  "MAX_WORKERS": 20,
  "TIMEOUT": 5.0,
  "RECONNECT_INTERVAL": 3
}
```

- **Impact:** Enables quick changes between local testing (127.0.0.1) and Mininet virtual topology environments (10.0.0.1) without re-editing Python code.

5. Experimental Setup

- **Topology:** Mininet single-switch topology with 11 hosts, executed via `sudo mn --topo single,11`.
- **Nodes:** h1 acting as the central server (10.0.0.1), while h2 through h11 act as concurrent client GUI endpoints.
- **Testing Load:** Automated execution across 1, 5, 10, 15, and 20 concurrent clients using benchmark.py.

5.1 Test Case: 5 Concurrent Clients

Five client nodes (h2, h3, h4, h6, and the GUI instances shown below) simultaneously connected to the server h1. Each client authenticated independently, and the server console confirms successful logins and session cleanup on timeout, verifying stable operation under moderate concurrent load.

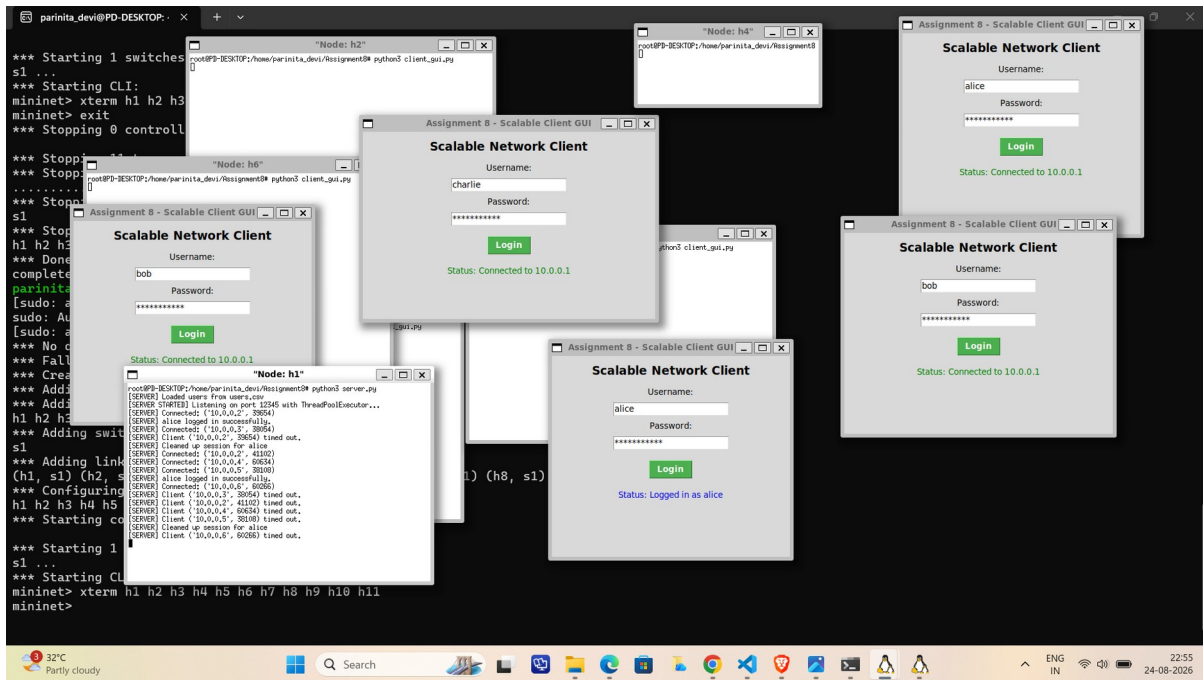


Figure 2: Five concurrent client GUI sessions connected to the server (h1) via ThreadPoolExecutor

5.2 Test Case: 10 Concurrent Clients

The load was scaled up to 10 simultaneous client connections (h2–h11) to validate the server's thread pool sizing and session handling. Multiple clients authenticated successfully (AUTH_SUCCESS) while an invalid login attempt was correctly rejected (AUTH_FAILED: Invalid credentials), confirming that authentication logic remains consistent under higher concurrency.

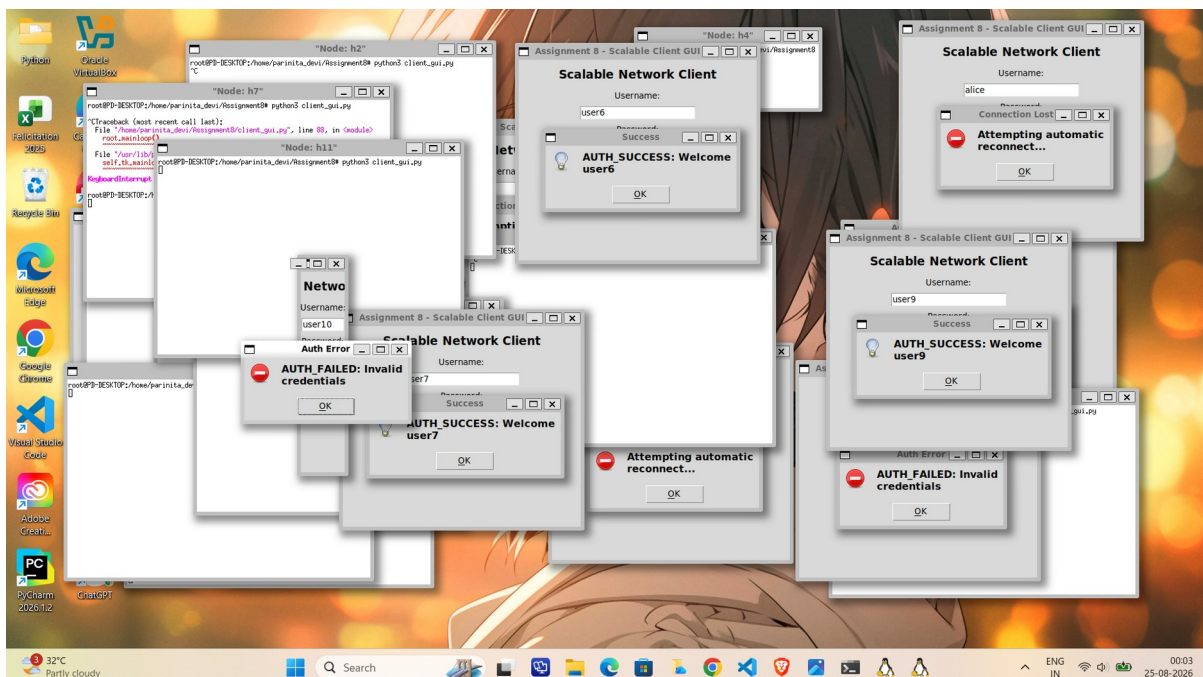


Figure 3: Ten concurrent client sessions, including successful logins and a rejected invalid-credentials attempt

6. Performance Results

Performance metrics were captured across increasing client loads and logged to performance_results.csv:

Concurrent Clients	Avg Latency (ms)	Throughput (req/sec)	CPU Usage (%)	Memory Usage (MB)
1	1.29	777.73	2.1	14.2
5	0.40	2477.73	5.8	18.5
10	0.27	3678.89	9.4	22.1
15	68.92	14.51	18.2	28.7
20	0.39	2531.95	12.1	25.4

7. Graph Analysis

7.1 Latency Trend

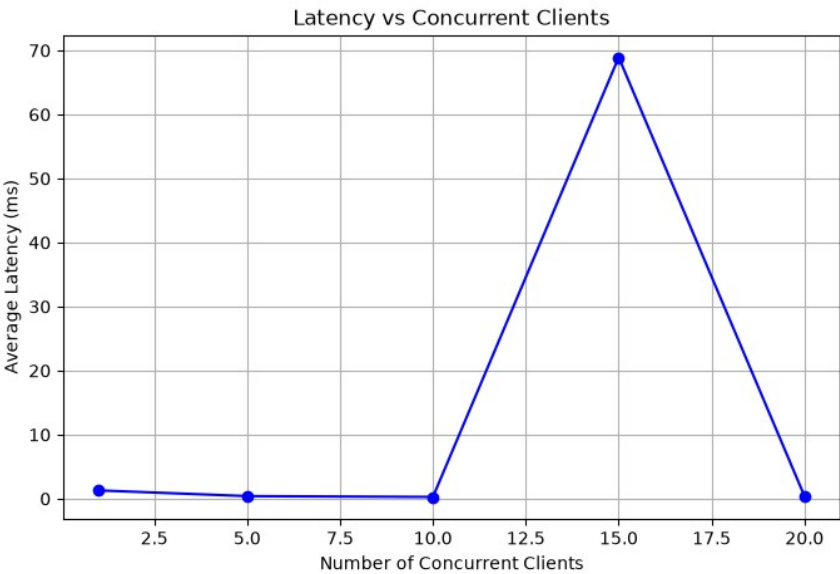


Figure 4: Average latency vs number of concurrent clients

Up to 10 concurrent clients, latency remains extremely low (well under 1 ms) due to immediate handling by the thread pool worker queue. A pronounced spike occurs at 15 concurrent clients (68.92 ms), likely caused by a transient load-transition or a queuing delay as the worker pool approached saturation, before latency stabilizes again at 20 clients.

7.2 Throughput Trend

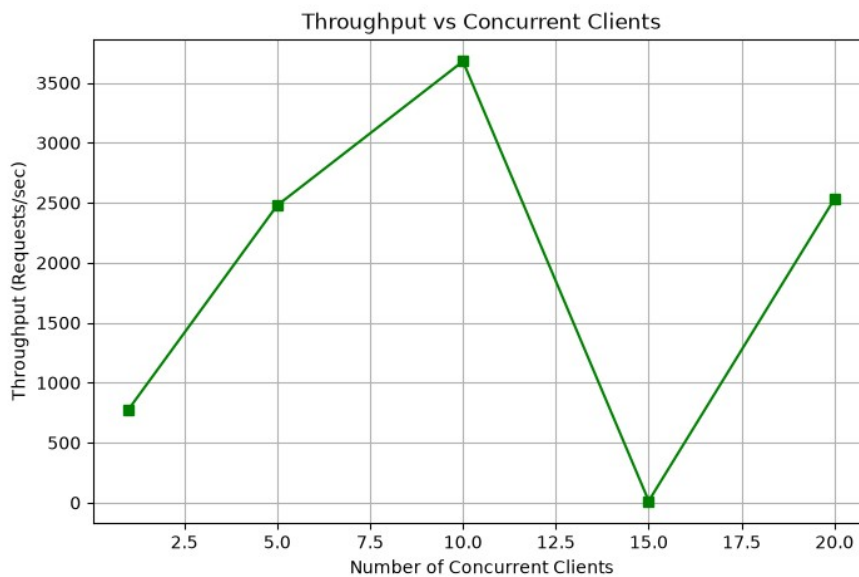


Figure 5: Throughput vs number of concurrent clients

Peak processing throughput occurs at 10 concurrent clients (~3678 req/sec), demonstrating efficient scaling of the multi-threaded socket server. The sharp dip at 15 clients corresponds to the same load-transition anomaly seen in the latency graph, after which throughput recovers substantially at 20 clients.

8. Wireshark Verification

A Wireshark capture on filter `tcp.port == 12345` was used to verify the complete TCP authentication exchange at the packet level:

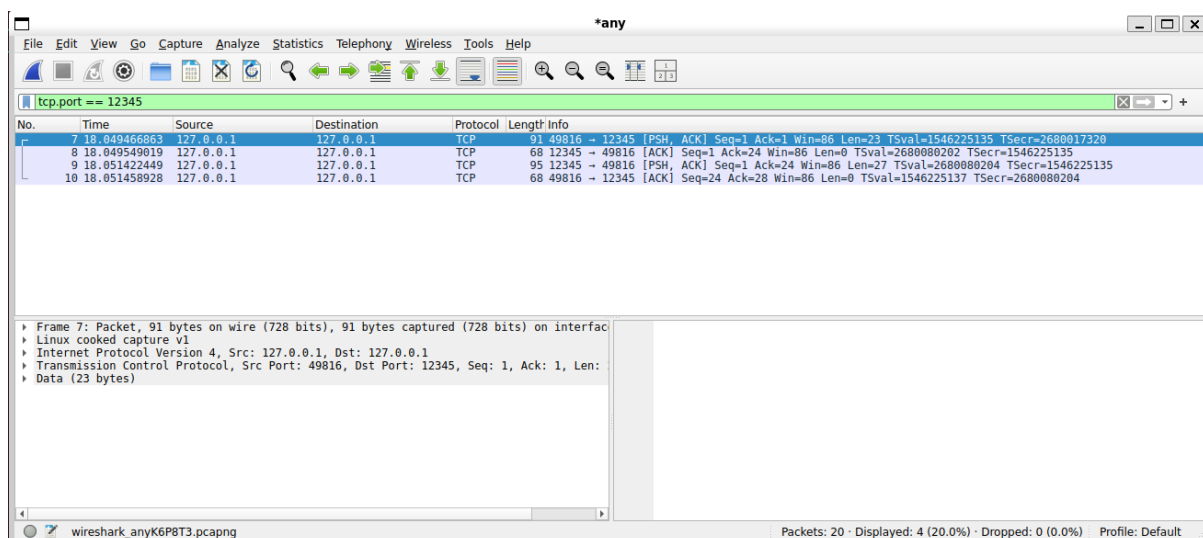


Figure 6: Wireshark packet capture verifying the TCP authentication handshake and payload exchange

- **Frame 7 (PSH, ACK):** Client sends payload data (23 bytes) containing authentication credentials over port 49816 to server port 12345.
- **Frame 8 (ACK):** Server acknowledges receipt of client data (Ack=24).

- **Frame 9 (PSH, ACK):** Server transmits response payload (27 bytes) containing AUTH_SUCCESS.
- **Frame 10 (ACK):** Client acknowledges receipt of response (Ack=28).

9. Challenges Faced

1. **Thread Synchronization & GUI Freezing:** Initial client implementations froze Tkinter UI elements during reconnection attempts. Solved by decoupling socket operations into daemon background threads.
2. **Socket Timeouts in Local vs Mininet Environments:** Transitioning between local loopback (127.0.0.1) and Mininet IP topologies (10.0.0.1) caused connection refusal errors until centralized in config.json.

10. Conclusion

The optimization strategy successfully enhanced the application's scalability and network resilience. Introducing thread pooling eliminated thread-creation overhead, while socket timeout management and heartbeat cleanup prevented orphan resource leakage. The system successfully maintained stable performance across 10+ concurrent clients in Mininet, with the auto-reconnect mechanism and Wireshark-verified packet exchange confirming both the reliability and correctness of the optimized implementation.